DSP in VLSI
Homework 1 - Sorter

Jen-Tse Chen

March 2026

1. First, generate several random sequences with 32 elements $[x_1, x_2, \ldots, x_{32}]$ in a block between -256 and 255 as your test inputs and prepare as the testbench. (10%)

   A Python script was used to generate random sequences of 32 elements with values ranging from -256 to 255. These sequences are stored in `input.txt`.

   The testbench reads this file to provide a continuous stream of input blocks for the simulation. For the corresponding signal transitions, please refer to the waveforms provided in Figure 6.

2. Please use Verilog to implement `Sort8` with bitonic sorter. Feed the 8 inputs $[x_1, x_2, \ldots, x_8]$ simultaneously in one clock cycle. Observe the outputs to realize sorting among eight elements in one group. Use $[x_1, x_2, \ldots, x_8]$ as the input and check for the correctness of the function "`Sort8`", which can generate sorted output from maximum to minimum. Indicate this function output in the timing diagram of your behavior simulation results. (If the data value inside the timing diagram can not be clearly seen, it will not be regarded as correct answers.) (10%)

   The functional correctness of the `Sort8` module was verified via behavioral simulation. Eight signed integers were fed simultaneously into the bitonic sorter within a single clock cycle. The simulation output (shown in Figure 1) matches the Golden Model generated by the Python script: `[254, 220, 126, 109, 66, -26, -158, -239]`.

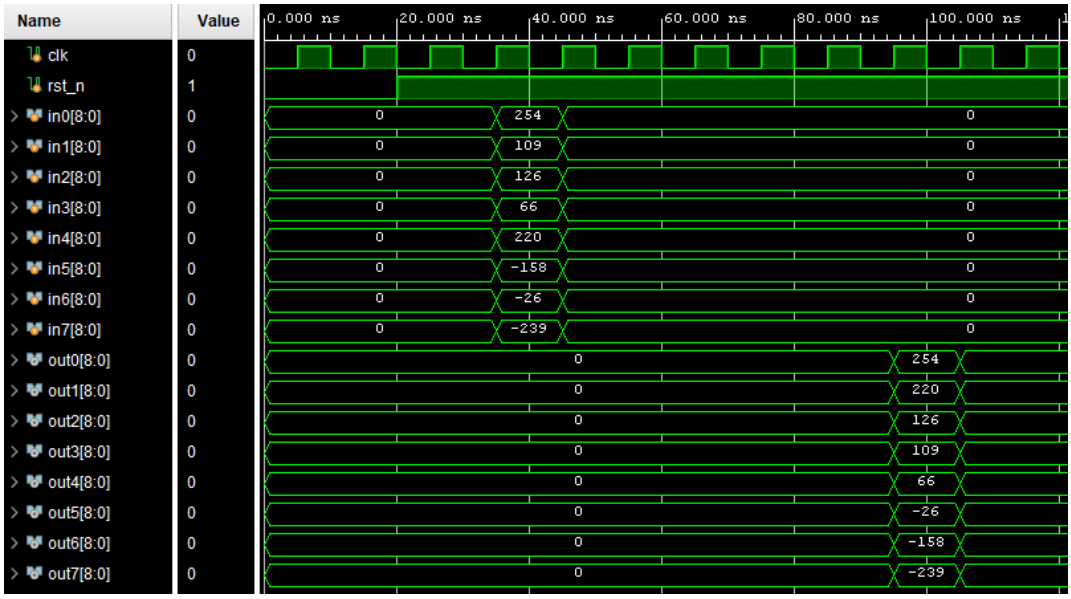

Figure 1: Verification of module `Sort8`

3. Construct the module (`SelectTopK`) of merger sort to select top 4 among 32 input elements ($[x_1, x_2, x_3, \ldots, x_{32}]$) under your control. Use a comparator tree to receive the sorted results from each group and then activate your comparator tree to select top 4 values. Maybe you need two sets of `Reg8` buffers, one for saving the outputs of the next block from `Sort8`, and the other for sending the contents of the current block to the comparator tree. Draw the block diagram of your own implementation and calculate the number of adders/subtractors/comparators in your block diagram. (20%)
   Show the timing diagram of your behavior simulation results (10%)
   Show the hardware output is correct by comparing it to your Matlab results. (10%)

   The design implements a hierarchical selection network to extract the top-4 elements from 32 inputs. Based on the architectural diagrams (Figures 2 through 5), the hardware resource consumption is as follows:

   (a) Module `Sort8` utilizes 24 9-bit comparators (based on the standard $N \log^2(N)$ stages for a bitonic network).

   (b) Module of comparator tree used to merge and select the top-4 values consists of 3 9-bit comparators.

   (c) The complete module `SelectTopK` utilizes 27 9-bit comparators.

   The behavioral simulation results, illustrated in Figure 6 and Figure 7, confirm the module's ability to process 32 elements across four distinct blocks. The hardware output was verified against the Golden Model generated by the Python script:

```
Block 0: [254 220 174 164]
Block 1: [248 225 195 189]
Block 2: [236 235 228 203]
Block 3: [248 233 219 217]
```

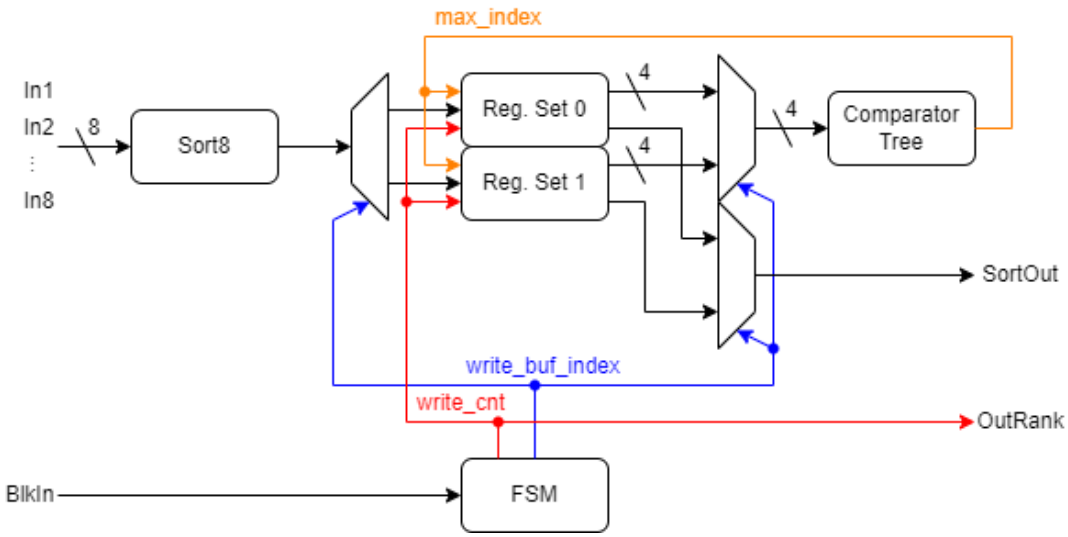

Figure 2: Design of module SelectTopK

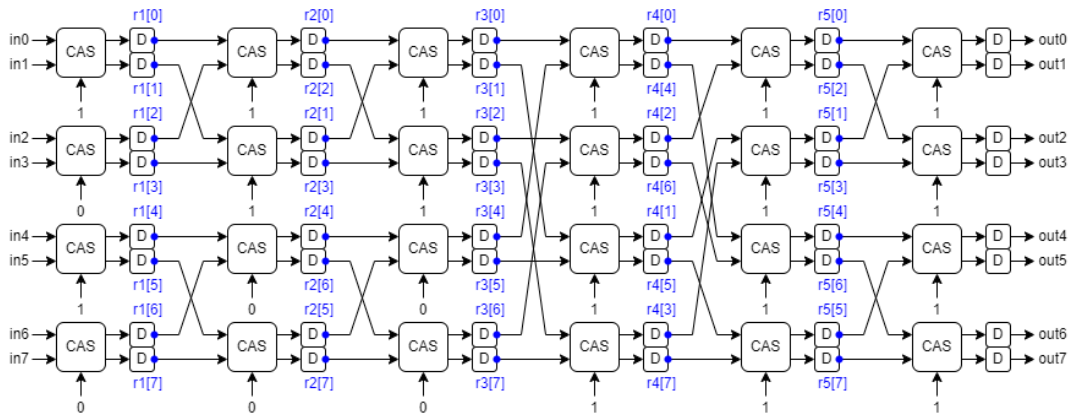

Figure 3: Design of module `Sort8` (CAS is Compare And Swap, refer to Figure 4 for details)

4. Show that your design can generate required top-4 output in descending order every 4 clock cycles with proper input signal "`BlkIn`" and output indicator "`OutRank`". Please feed 4 blocks and check the outputs of four blocks. If it is too long, please cut it down into several segments to make the numerical expressions in the timing diagram clear. If the numerical expressions are hard to distinguish, correct evaluation may not be given. (30%)

   As shown in Figures 6 and 7, the design successfully generates the top-4 output in descending order every 4 clock cycles. The `BlkIn` signal effectively triggers block processing, while `OutRank` correctly identifies the sorted position of each element.

   To sustain continuous streaming input, the design implements:

   (a) Ping-Pong Buffering: A dual-buffer scheme (Fig. 2) allows simultaneous sorting of new data while selecting the Top-4 from the previous set.

   (b) FSM Control: A Finite State Machine (Fig. 8) orchestrates the buffer switching and signal timing, ensuring a seamless transition across the 4-block sequence. Upon receiving the `BlkIn` signal, the `blk_cnt` register increments. Conversely, once a block output operation completes, `blk_cnt` decrements. The ping-pong buffering state machine transitions back to IDLE only when `blk_cnt` reaches zero.

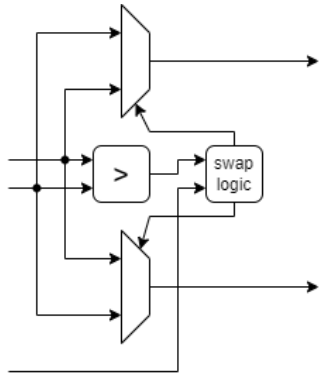

Figure 4: Design of module of compare and swap

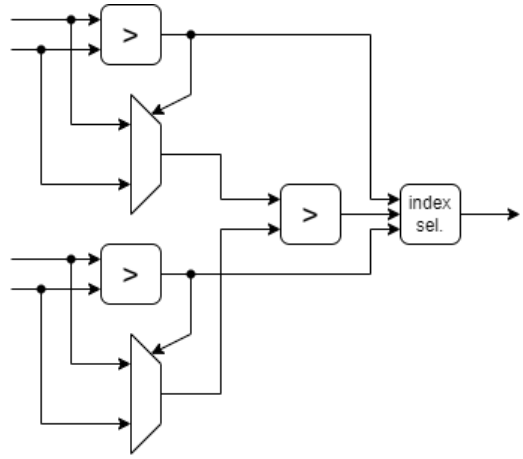

Figure 5: Design of module of comparator tree

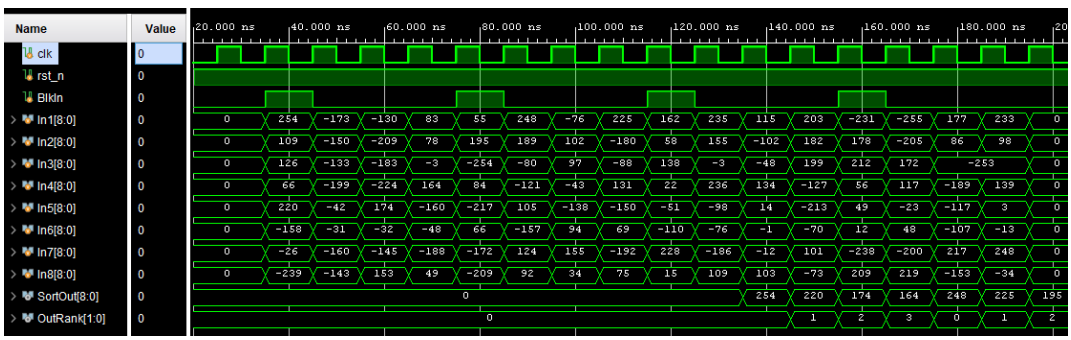

Figure 6: Verification input of module `SelectTopK`

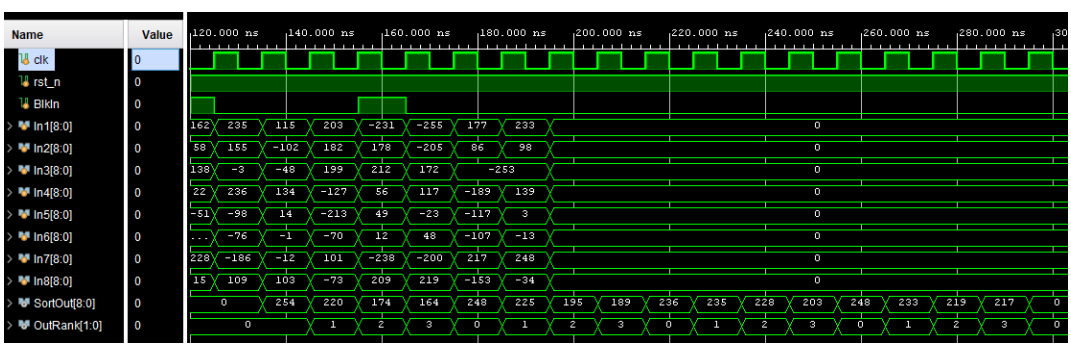

Figure 7: Verification output of module `SelectTopK`

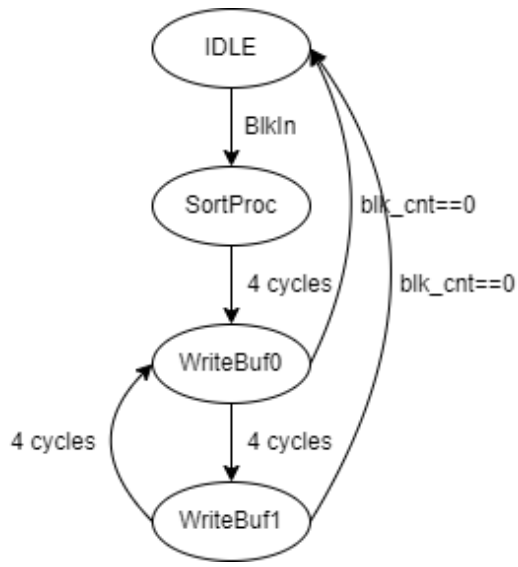

Figure 8: Finite state machine of module `SelectTopK`

5. Please use Matlab command "sort" to verify your results of your randomly-generated sequence and compare to the Verilog simulation results (10%).

   The functional correctness of the design is verified against the randomly generated input and the Golden Model described in Q3. Both methods confirm that the hardware output is perfectly aligned with the expected sorted results.

6. Synthesis your design in Q4. Show the number of adders/subtractors/ comparators in your design. Sum them up together to see if it matches with your block diagram. (10%) (Because you may use counters to control your circuits, we will not ask that the two values should be exactly the same. However, there should not be a large discrepancy.)

   The design was synthesized at a clock frequency of 50 MHz. The resulting timing and area reports are provided in Figure 9 and Figure 10, respectively.

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	6.017 ns	Worst Hold Slack (WHS):	0.063 ns	Worst Pulse Width Slack (WPWS):	9.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	1552	Total Number of Endpoints:	1552	Total Number of Endpoints:	1033

All user specified timing constraints are met.

Figure 9: Timing report

Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice (33650)	LUT as Logic (134600)	Bonded IOB (400)	BUFGCTRL (32)
SelectTopK	907	1032	70	35	325	907	86	1
fsm0 (merge_sort_fsm)	426	12	0	35	178	426	0	0
max0 (max_index_finder)	144	0	70	0	44	144	0	0
sort0 (Sort8)	337	432	0	0	114	337	0	0

Figure 10: Area report

The architectural design specifies 27 9-bit comparators (24 within the `Sort8` module and 3 within the module of comparator tree). In the targeted FPGA architecture, a single 9-bit signed comparison typically maps to 2 `CARRY4` primitives. The synthesis report indicates a total usage of 54 `CARRY4` blocks. This matches the expected consumption, confirming that the synthesizer preserved the intended logic structure without significant discrepancy.

Ref Name	Used	Functional Category
FDCE	1030	Flop & Latch
LUT3	447	LUT
LUT4	444	LUT
LUT6	237	LUT
LUT5	83	LUT
IBUF	75	IO
MUXF7	70	MuxFx
LUT2	62	LUT
CARRY4	54	CarryLogic
MUXF8	35	MuxFx
OBUF	11	IO
FDPE	2	Flop & Latch
LUT1	1	LUT
BUFG	1	Clock